

1. Fundamental Concept Of Applet:

An applet is a small Java program that can be run as a part of web document under Java-compatible web browser or Sun's applet viewer program called appletviewer. All applets are the subclass of APPLET class so we need to import the applet package. **An applet program is written as an inheritance of the java.Applet class. Since applets run in a window, it is important to include support for windows. So, applets must also import java.awt.*. An applet uses AWT for graphics. There is no main() method in an Applet.** Applets are designed to be embedded within an HTML page.

2. Life Cycle of an Applet:

Four methods in the Applet class give you the framework on which you build any serious applet:

- **init:** This method is the first method to be called. It is used for various initialization needed for an applet. It is called only once during runtime of our applet. It is called after the param tags have been processed by the JVM.
- **start:** This method is automatically called after the browser calls the init() method. It is also called whenever the user returns to the page containing the applet after visiting other pages. It is called when applet is loaded.
- **stop:** This method is automatically called when the user moves away from the page containing the applet. It can be called multiple times within the same applet.
- **destroy:** This method is only called when the browser shuts down normally. It deallocates the applet's resources and removes the applet completely from the memory.
- **paint:** it is invoked immediately after the start() method, and also any time the applet needs to repaint itself in the browser. The paint() method is actually inherited from the java.awt.

Every Applet needs to implement one or more of the init(), the start() and the paint() methods. At the end of the execution, the stop() method is invoked, followed by the destroy() method to deallocate the applet's resources.

3. Simple Applet

➤ A "Hello, World" Applet:

The following is a simple applet named HelloWorldApplet.

```
import java.applet.*;
import java.awt.*;
public class Hello extends Applet
{
    public void paint (Graphics g)
    {
        g.drawString ("Hello World", 25, 50);
    }
}
```

Save this file as Hello.java.

These import statements bring the classes into the scope of our applet class:

```
java.applet.Applet.
java.awt.Graphics.
```

Without these import statements, the Java compiler would not recognize the classes Applet and Graphics, which the applet class refers to.

➤ Invoking an Applet:

An applet may be invoked by embedding directives in an HTML file and viewing the file through an applet viewer or Java-enabled browser. The <applet> tag is the basis for embedding an applet in an HTML file. Below is an example that invokes the "Hello, World" applet:

```
<html>
<title>The Hello, World Applet</title>
<hr>
<applet code="HelloWorldApplet.class" width="320" height="120">
</applet>
<hr>
</html>
```

Save this as Hello.html

If our browser is Java-enabled, a "Hello World" message will be displayed.

Note: We can refer to HTML Applet Tag to understand more about calling applet from HTML. The code attribute of the <applet> tag is required. It specifies the Applet class to run. Width and height are also required to specify the initial size of the panel in which an applet runs. The applet directive must be closed with a </applet> tag.

If an applet takes parameters, values may be passed for the parameters by adding <param> tags between <applet> and </applet>. The browser ignores text and other tags between the applet tags. Non-Java-enabled browsers do not process <applet> and </applet>. Therefore, anything that appears between the tags, not related to the applet, is visible in non-Java-enabled browsers.

If an applet resides in a package other than the default, the holding package must be specified in the code attribute using the period character (.) to separate package/class components. For example:
<applet code="mypackage.subpackage.TestApplet.class"
width="320" height="120">

➤ **Steps to be done to writing and running an applet program**

- Edit java source code & html in notepad as Hello.java and Hello.html
- Compile source code to Byte Codes as javac Hello.java and it produces Hello.class
- View applet (Java Virtual Machine) as:

Type appletviewer Hello.html in command prompt for viewing using applet viewer

Or

Open Hello.html for viewing using the browser.

4. Applets HTML Tags and Attributes

The HTML <applet> tag specifies an applet. The APPLET tag be used to start an applet from both an HTML document and from an applet viewer. The syntax of the APPLET tag is shown here. Bracketed items are optional.

< APPLET

[CODEBASE = codebaseURL]

CODE = appletFile

[ALT = alternateText]

[NAME = appletInstanceName]

WIDTH = pixels HEIGHT = pixels

[ALIGN = alignment]

Lecture Notes By Rashmi Paneru Pandey(BE Computer)

[VSPACE = pixels] [HSPACE = pixels]

>

...

[HTML Displayed in the absence of Java]

</APPLET>

Attribute	Value	Description
align	URL	ALIGN is an optional attribute that specifies the text alignment of the applet.
alt	URL	The ALT tag is an optional attribute used to specify a short text message that should be displayed if the browser recognizes the APPLET tag but can't currently run Java applets. Or it is the alternate text to be displayed in case browser does not support applet.
code	URL	CODE is a required attribute that gives the name of the file containing our applet's compiled .class file i.e. a URL that points to the class of the applet
codebase	URL	CODEBASE is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file (specified by the CODE tag). The HTML document's URL directory is used as the CODEBASE if this attribute is not specified.
Width and height	pixels	WIDTH and HEIGHT are required attributes that give the size (in pixels) of the applet display area.
hspace	pixels	HSPACE specifies the space, in pixels, on each side of the applet i.e. it defines the left and right spacing around the applet.
name	name	NAME is an optional attribute used to specify a unique name for the applet. Applets must be named in order for other applets on the same page to find them by name and communicate with them. To obtain an applet by name, we use getApplet() , which is defined by the AppletContext interface.
title	text	Additional information to be displayed in tool tip of the mouse
vspace	pixels	VSPACE specifies the space, in pixels, above and below the applet.

5. Testing

To view and test an applet more conveniently, we simply include a comment at the head of our Java source code file that contains the APPLET tag. In this way, our code is documented with the necessary HTML statements needed by our applet, and we can test the compiled applet by starting the applet viewer with our Java source code file specified as the target. Here is an example of such a comment:

```
/*
<applet code="MyApplet" width=200 height=60>
</applet>
*/
```

This comment contains an APPLET tag that will run an applet called **MyApplet** in a window that is 200 pixels wide and 60 pixels high. Because the inclusion of an APPLET command makes testing applets easier, so we can write all the applets using the appropriate APPLET tag embedded in a comment.

Example:

```
import java.applet.*;
import java.awt.*;
/*
<applet code="MyApplet" width=200 height=60>
</applet>
*/
public class MyApplet extends Applet
{
    public void paint (Graphics g)
    {
        g.drawString ("Applet Testing Example", 20, 30);
    }
}
```

➤ **Steps For Executing the java class in which the Applet tag is embedded inside the comment line**

- Edit java source file and save it as MyApplet.java
- Compile the java file as javac MyApplet.java
- Execute the applet viewer, specifying the name of the applet's source file as:
appletviewer MyApplet.java

The applet viewer will encounter the applet tag within the comment and execute the applet.

Method

void destroy()

Description

Called by the browser just before an applet is terminated. Our applet will override this method if it needs to perform any cleanup prior to its destruction.

AppletContext getAppletContext()

Returns the context associated with the applet.

String getAppletInfo()

Returns a string that describes the applet.

AudioClip getAudioClip(URL url)

Returns an AudioClip object that encapsulates the audio clip found at the location specified by url.

AudioClip getAudioClip(URL url,
String clipName)

Returns an AudioClip object that encapsulates the audio clip found at the location specified by url and

Lecture Notes By Rashmi Paneru Pandey(BE Computer)

	having the name specified by clipName.
URL getCodeBase()	Returns the URL associated with the invoking applet.
URL getDocumentBase()	Returns the URL of the HTML document that invokes the applet.
Image getImage(URL url)	Returns an Image object that encapsulates the image found at the location specified by url.
Image getImage(URL url, String imageName)	Returns an Image object that encapsulates the image found at the location specified by url and having the name specified by imageName.
String getParameter(String paramName)	Returns the parameter associated with paramName. null is returned if the specified parameter is not found.
String[][] getParameterInfo()	Returns a String table that describes the parameters recognized by the applet. Each entry in the table must consist of three strings that contain the name of the parameter, a description of its type and/or range, and an explanation of its purpose.
void init()	Called when an applet begins execution. It is the first method called for any applet.
boolean isActive()	Returns true if the applet has been started. It returns false if the applet has been stopped.
static final AudioClip newAudioClip(URL url)	Returns an AudioClip object that encapsulates the audio clip found at the location specified by url. This method is similar to getAudioClip() except that it is static and can be executed without the need for an Applet object.
void play(URL url)	If an audio clip is found at the location specified by url, the clip is played.
void play(URL url, String clipName)	If an audio clip is found at the location specified by url with the name specified by clipName, the clip is played.
void resize(Dimension dim)	Resizes the applet according to the dimensions specified by dim. Dimension is a class stored inside java.awt. It contains two integer fields: width and height.
void resize(int width, int height)	Resizes the applet according to the dimensions specified by width and height.
void showStatus(String str)	Displays str in the status window of the browser or applet viewer. If the browser does not support a status window, then no action takes place.

<code>void start()</code>	Called by the browser when an applet should start (or resume) execution. It is automatically called after <code>init()</code> when an applet first begins.
<code>void stop()</code>	Called by the browser to suspend execution of the applet. Once stopped, an applet is restarted when the browser calls <code>start()</code> .

Example that sets the background color to green and the text color to red:

```
setBackground(Color.green);
setForeground(Color.red);
```

A good place to set the foreground and background colors is in the **init()** method. We can change these colors as often as necessary during the execution of our applet. We can obtain the current settings for the background and foreground colors by calling **getBackground()** and **getForeground()**, respectively. They are also defined by **Component** and are shown below:

```
Color getBackground( )
Color getForeground( )
```

Here is a very simple applet that sets the background color to green, the foreground color to red, and displays a message showing the order of **init()**, **start()**, and **paint()** methods:

```
import java.awt.*;
import java.applet.*;
/*
<applet code="Sample" width=300 height=50>
</applet>
*/
public class Sample extends Applet{
String msg;
// set the foreground and background colors.
public void init() {
setBackground(Color.green);
setForeground(Color.red);
msg = "Inside init( ) --";
}
// Initialize the string to be displayed.
public void start() {
msg += " Inside start( ) --";
}
// Display msg in applet window.
public void paint(Graphics g) {
msg += " Inside paint( ).";
g.drawString(msg, 10, 30);
}
}
```

The methods **stop()** and **destroy()** are not overridden, because they are not needed by this simple applet.

Example Using the Status Window

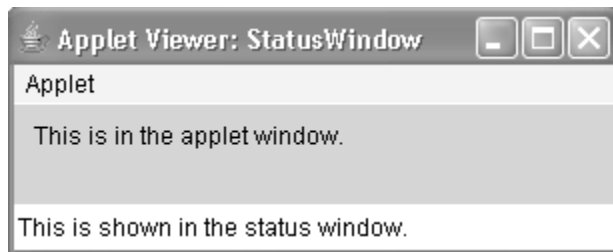
In addition to displaying information in its window, an applet can also output a message to the status window of the browser or applet viewer on which it is running. For doing this, it call **showStatus()**

with the string that we want to display. In the status window we can give the feedback about what is occurring in the applet, suggest options, or possibly report some types of errors.

Example:

```
import java.awt.*;
import java.applet.*;
/*
<applet code="StatusWindow" width=300 height=50>
</applet>
*/
public class StatusWindow extends Applet
{
    public void init()
    {
        setBackground(Color.cyan);
    }
    // Display msg in applet window.
    public void paint(Graphics g)
    {
        g.drawString("This is in the applet window.", 10, 20);
        showStatus("This is shown in the status window.");
    }
}
```

Sample output from this program is shown here:



6. Application Conversion to Applets:

It is easy to convert a graphical Java application (i.e. an application that uses the AWT and that we can start with the java program launcher) into an applet that we can embed in a web page. Here are the specific steps for converting an application to an applet.

- Make an HTML page with the appropriate tag to load the applet code.
- Extend the JApplet (Applet) class. Make this class public. Otherwise, the applet cannot be loaded.
- Eliminate the main method in the application. Do not construct a frame window for the application. Our application will be displayed inside the browser.
- Move any initialization code from the frame window constructor to the init() method of the applet. We don't need to explicitly construct the applet object. The browser instantiates it and calls the init() method.
- Remove the call to setSize; for applets, sizing is done with the width and height parameters in the HTML file.
- Remove the call to setDefaultCloseOperation. An applet cannot be closed; it terminates when the browser exits.
- If the application calls setTitle, eliminate the call to the method. Applets cannot have title bars. (We can title the web page using the HTML title tag.)
- Don't call setVisible(true). The applet is displayed automatically.

7. Passing Parameters to Applets

It is possible to send information from an HTML page to an applet. The information is passed in HTML parameter tags and is retrieved in the applet's code. The PARAM tag allows us to specify applet-specific arguments in an HTML page. Applets access their attributes with the **getParameter()** method. At any point within the applet, the method `getParameter` can retrieve the parameter's value, but always as a string. Thus, for numeric and **boolean** values, we will need to convert their string representations into their internal formats. If there are several parameters, each requires its own tag.

Example:

```
import java.awt.*;
import java.applet.*;
/*
<applet code="ParamDemo" width=300 height=80>
<param name=fontName value=Courier>
<param name=fontSize value=14>
<param name=active value=true>
</applet>
*/
public class ParamDemo extends Applet{
String fontName;
int fontSize;
boolean active;
// Initialize the string to be displayed.
public void start() {
String param;
fontName = getParameter("fontName");
if(fontName == null)
fontName = "Not Found";
param = getParameter("fontSize");
try {
if(param != null)
fontSize = Integer.parseInt(param);
else
fontSize = 0;
} catch(NumberFormatException e) {
fontSize = -1;
}
param = getParameter("active");
if(param != null)
active = Boolean.valueOf(param).booleanValue();// or Boolean.parseBoolean(param);
}
// Display parameters.
public void paint(Graphics g) {
g.drawString("Font name: " + fontName, 0, 10);
g.drawString("Font size: " + fontSize, 0, 26);
g.drawString("Active: " + active, 0, 58);
}
}
```


As the program shows, we should test the return values from **getParameter()**. If a parameter isn't available, **getParameter()** will return **null**. Also, conversions to numeric types must be attempted in a **try** statement that catches **NumberFormatException**. Uncaught exceptions should never occur within an applet.

8. Images in Applets

Applets can be used to browse and draw images and animations. To draw the images, the **java.awt.Graphics** class comes with a method **drawImage()**. With the help of this method and **ImageObserver** at the background, images can be drawn on the applet window.

Example:

```
import java.awt.*;
import java.applet.*;
import java.net.*;
public class ImageWithApplet extends Applet
{
    public void paint( Graphics g )
    {
        URL url1 = getCodeBase();
        Image img = getImage(url1, "image.gif");
        g.drawImage(img, 60, 120, this);
    }
}
```

HTML file to run the above applet – ImageWithApplet.html

```
<applet code="ImageWithApplet.class" width="350" height="350">
</applet>
```

Any image to be drawn should be converted into an object of **Image** class. The **getImage()** method of Applet class returns an object of Image, (in this case **img**). This method takes two parameters – the address of the image (image.gif) as an object of URL and the image itself as a string. We should place the image in the same folder where the applet exists. **getCodeBase()** method of Applet class returns an object of URL (here, **url1**) that contains the address of the image (as image and applet exist in the same folder).

Graphics is an abstract class from **java.awt** package and contains many methods of drawing. One of such methods is **drawImage()** that takes four parameters. The first parameter is the image to be drawn as an object of **Image** class. The second and third parameters are **x** and **y** coordinates where the image is to be drawn on the applet window. The last parameter is an object of **ImageObserver**. The "**this**" argument in the above statement refers an object of ImageObserver. The super class of Applet, **java.awt.Component** class implements the ImageObserver.

9. Playing Audio:

An applet can play an audio file represented by the AudioClip interface in the java.applet package. The AudioClip interface has three methods, including:

public void play(): Plays the audio clip one time, from the beginning.

public void loop(): Causes the audio clip to replay continually.

public void stop(): Stops playing the audio clip.

To obtain an AudioClip object, we must invoke the **getAudioClip()** method of the Applet class. The **getAudioClip()** method returns immediately, whether or not the URL resolves to an actual audio file.

Lecture Notes By Rashmi Paneru Pandey(BE Computer)

The audio file is not downloaded until an attempt is made to play the audio clip. Following example shows all the steps to play an audio:

```
import java.applet.*;
import java.awt.*;
import java.net.*;
/*
<applet code="AudioDemo.class" width="0" height="0">
<param name="audio" value="test.wav">
</applet>
*/
public class AudioDemo extends Applet
{
    private AudioClip clip;
    private AppletContext context;
    public void init()
    {
        context = this.getAppletContext();
        String audioURL = this.getParameter("audio");
        if(audioURL == null)
        {
            audioURL = "default.au";
        }
        try
        {
            URL url = new URL(this.getDocumentBase(), audioURL);
            clip = context.getAudioClip(url);
        } catch (MalformedURLException e)
        {
            e.printStackTrace();
            context.showStatus("Could not load audio file!");
        }
    }
    public void start()
    {
        if(clip != null)
        {
            clip.loop();
        }
    }
    public void stop()
    {
        if(clip != null)
        {
            clip.stop();
        }
    }
}
```

10.Event Handling:

Applets inherit a group of event-handling methods from the Container class. The Container class defines several methods, such as processKeyEvent and processMouseEvent, for handling particular

Lecture Notes By Rashmi Paneru Pandey(BE Computer)

types of events, and then one catch-all method called `processEvent`. In order to react an event, an applet must override the appropriate event-specific method.

Example:

```
import java.awt.event.*;
import java.applet.*;
import java.awt.*;
/*
<applet code="ExampleEventHandling.class" width="300" height="300">
*/
public class ExampleEventHandling extends Applet implements MouseListener {
    public void init() {
        addMouseListener(this);
        StringBuffer strBuffer = new StringBuffer();
        addItem("initializing the apple ");
    }
    public void start() {
        addItem("starting the applet ");
    }
    void addItem(String word) {
        System.out.println(word);
        strBuffer.append(word);
        repaint();
    }
    public void paint(Graphics g) {
        //Draw a Rectangle around the applet's display area.
        g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);
        //display the string inside the rectangle.
        g.drawString(strBuffer.toString(), 10, 20);
    }
    public void mouseClicked(MouseEvent event) {
        addItem("mouse clicked");
    }
}
```

Initially, the applet will display "initializing the applet. Starting the applet." Then once we click inside the rectangle "mouse clicked" will be displayed as well.